THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

OBJECT ORIENTED PROGRAMMING

Assignment No.4

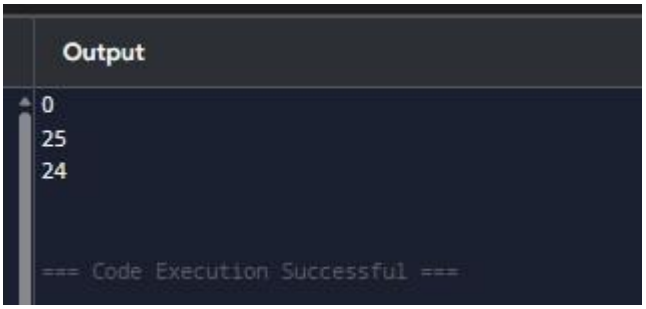
SUBMITTED BY: RIDHIKA BANSAL

1024150280

2O31

1. Create a class named Rectangle with two data members- length and breadth and a function to calculate the area which is length*breadth . The class has three constructors which are : (a) having no parameter - values of both length and breadth are assigned zero.
(b) having two numbers as parameters - the two numbers are assigned as length and breadth respectively.
(c) having one number as parameter - both length and breadth are assigned that number.
Now, create objects of the Rectangle class having none, one and two parameters and print their areas.

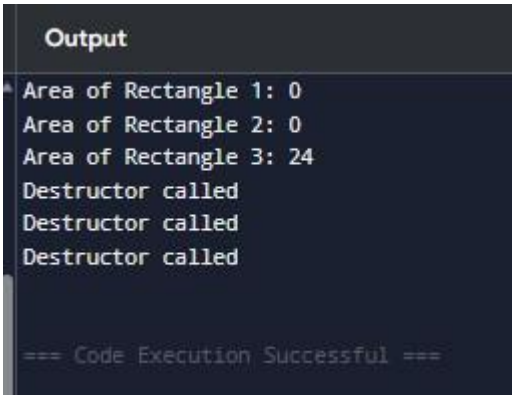
```
#include <iostream>
using namespace std;

class Rectangle
{ private:
   float length;     float breadth;

public:
   Rectangle()
   {        length = 0;         breadth = 0;
   }

   Rectangle(float l, float b)
   {        length
= l;        breadth
= b;
   }

   Rectangle(float s)
   {        length
= s;        breadth
= s;
   }

   float area()
   {
      return length * breadth;
   }
};
```



```
int main()
{
   Rectangle r1;     cout <<
r1.area() << endl;

   Rectangle r2(5);     cout
<< r2.area() << endl;

   Rectangle r3(4, 6);
cout << r3.area() << endl;

   return 0;
}
```

2. Redefine the above program by creating an array of objects of the class Rectangle and calculate area for each object calling different constructors. Also implement constructors with default arguments and destructor in this program.

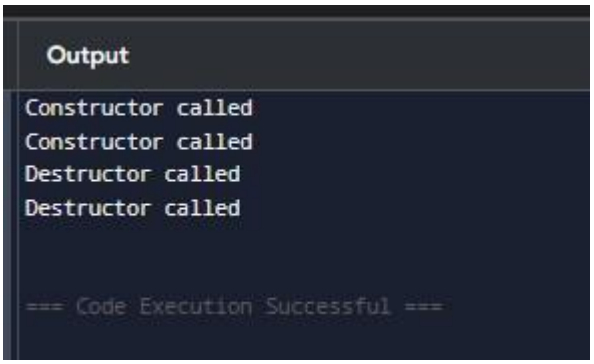
```
#include <iostream>
using namespace std;

class Rectangle
{ private:
   float length;     float breadth;

public:
   Rectangle(float l = 0, float b = 0)
   {        length = l;        breadth = b;
   }

   float area()
   {        return length *
breadth;
   }

   ~Rectangle()
   {
      cout << "Destructor called" << endl;
   }
};

int main()
```



```
{
   Rectangle r[3] = { Rectangle(), Rectangle(5), Rectangle(4,6) };

   for(int i = 0; i < 3; i++)
   {
      cout << "Area of Rectangle " << i+1 << ": " << r[i].area() << endl;
   }

   return 0;
}
```

3. Verify the following about destructor by writing the program:
(i) Name should begin with tilde sign(~) and must match class name.
(ii) There cannot be more than one destructor in a class.
(iii) Destructors do not allow any parameter.
(iv) They do not have any return type, just like constructors. When you do not specify any destructor in a class, compiler generates a default destructor and inserts it into your code.

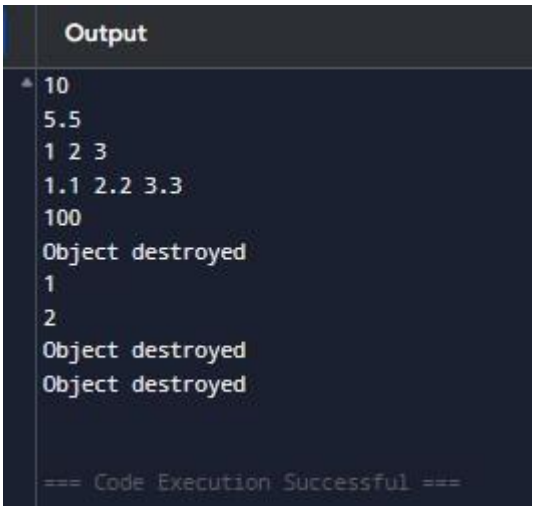
```
#include <iostream>
using namespace std;

class Demo
{ public:
   Demo()
   {
      cout << "Constructor called" << endl;
   }

   ~Demo()
   {
      cout << "Destructor called" << endl;
   }
};

int main() {
   Demo obj1;

   {
      Demo obj2;
   }
```



```
    return 0;
}
```

4. Implement dynamic memory allocation. Use new and delete keywords. (For integer variable, float variable, integer array, float array, class objects, Array of Objects)

```
#include <iostream>
using namespace std;

class Sample { public:
    int x;

    Sample(int a = 0)
    {       x = a;
    }

    void display()
    {       cout << x << endl;
    }

    ~Sample()
    {
        cout << "Object destroyed" << endl;
    }
};

int main()
{    int *pInt = new int;
*pInt = 10;     cout <<
*pInt << endl;     delete
pInt;

    float *pFloat = new float;
*pFloat = 5.5;     cout <<
*pFloat << endl;     delete
pFloat;

    int *arrInt = new int[3];
for(int i = 0; i < 3; i++)
arrInt[i] = i + 1;     for(int i =
0; i < 3; i++)        cout <<
```



```
arrInt[i] << " ";     cout <<
endl;     delete[] arrInt;

    float *arrFloat = new float[3];
for(int i = 0; i < 3; i++)
arrFloat[i] = 1.1f * (i + 1);
for(int i = 0; i < 3; i++)
cout << arrFloat[i] << " ";
cout << endl;     delete[]
arrFloat;

    Sample *obj = new Sample(100);
obj->display();     delete obj;

    Sample *objArr = new Sample[2]{ Sample(1), Sample(2) };
for(int i = 0; i < 2; i++)         objArr[i].display();     delete[]
objArr;

    return 0;
}
```